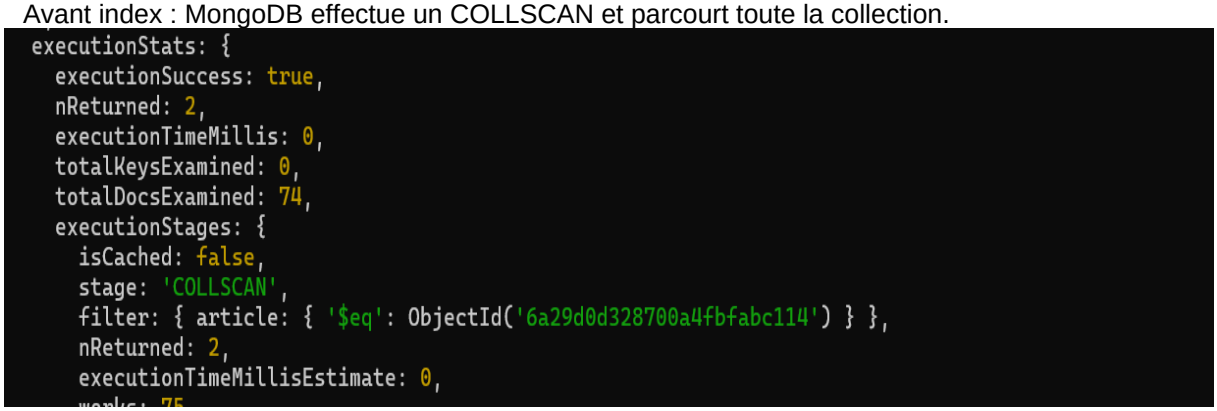
Analyse des performances : COLLSCAN vs IXSCAN

Avant index : MongoDB effectue un COLLSCAN et parcourt toute la collection.



```
executionStats: {
  executionSuccess: true,
  nReturned: 2,
  executionTimeMillis: 0,
  totalKeysExamined: 0,
  totalDocsExamined: 74,
  executionStages: {
    isCached: false,
    stage: 'COLLSCAN',
    filter: { article: { '$eq': ObjectId('6a29d0d328700a4fbfabc114') } },
    nReturned: 2,
    executionTimeMillisEstimate: 0,
```

Apres index : MongoDB utilise IXSCAN puis FETCH pour acceder uniquement aux documents necessaires.

```
executionStats: {
  executionSuccess: true,
  nReturned: 2,
  executionTimeMillis: 1,
  totalKeysExamined: 2,
  totalDocsExamined: 2,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 2,
    executionTimeMillisEstimate: 1,
    works: 3,
    advanced: 2,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 2,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 2,
      executionTimeMillisEstimate: 1,
      works: 3,
      advanced: 2,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { article: 1 },
      indexName: 'article_1',
      isMultiKey: false,
      multiKeyPaths: { article: [] },
      isUnique: false,
      isSparse: false,
```

Critere	Sans index	Avec index
Plan	COLLSCAN	FETCH -> IXSCAN
Docs examines	74	2
Cles examinees	0	2
Documents retournes	2	2

Conclusion : Sans index, MongoDB lit les 74 documents de la collection pour retrouver seulement 2 resultats. Apres creation de l'index article_1, le moteur utilise IXSCAN, examine uniquement 2 cles et 2 documents. L'indexation reduit fortement le nombre de lectures et ameliore les performances lorsque le volume de donnees augmente.